

House Price Prediction Using Machine Learning

Machine Learning Project

Anurag Kumar Poddar

*Data Analytics | Machine Learning |
Python*

Project Overview

- This project focuses on predicting house prices using supervised machine learning regression algorithms.
- The project explores housing data, prepares features for model training, and develops multiple regression models to estimate house prices. Their performance is evaluated and compared using standard regression metrics to identify the most suitable model for the dataset.
- **Project Goal: Build, evaluate, and compare machine learning regression models for house price prediction using Python and Scikit-learn.**



Project Objectives

- **Explore the Dataset:** Understand the dataset structure, features, and patterns.
- **Data Preprocessing:** Inspect missing values, examine data quality, and prepare categorical variables.
- **Feature Preparation:** Separate input features from the target variable.
- **Train-Test Split:** Divide the dataset into training and testing sets.
- **Model Development:** Train and compare Linear Regression, Ridge Regression, Decision Tree Regressor, and Random Forest Regressor.
- **Model Evaluation:** Measure performance using MAE, MSE, RMSE, and R^2 Score.
- **Data Visualization:** Compare actual and predicted house prices and interpret model performance.

Tools & Technologies



Tools & Technologies ¶

Tool	Purpose
Python	Programming and model development
Jupyter Notebook	Interactive coding and documentation
Pandas & NumPy	Data manipulation and numerical operations
Scikit-learn	Model training, prediction, and evaluation
Matplotlib & Seaborn	Data visualization

Machine Learning Approach

Component	Description
Learning Type	Supervised Learning
Problem Type	Regression
Target Variable	Price
Algorithms	Linear Regression, Ridge Regression, Decision Tree Regressor, Random Forest Regressor
Data Split	80% Training / 20% Testing
Evaluation Metrics	MAE, MSE, RMSE, R ² Score

Data Set- housing_price_dataset

- Room Area- Square feet
- Bedrooms
- Bathrooms
- Neighborhood
- YearBuilt
- Price

1	SquareFeet	Bedrooms	Bathrooms	Neighborhood	YearBuilt	Price
2	2126	4	1	Rural	1969	215355.2836
3	2459	3	2	Rural	1980	195014.2216
4	1860	2	1	Suburb	1970	306891.0121
5	2294	2	1	Urban	1996	206786.7872
6	2130	5	2	Suburb	2001	272436.2391
7	2095	2	3	Suburb	2020	198208.8039
8	2724	2	1	Suburb	1993	343429.3191
9	2044	4	3	Rural	1957	184992.3213
10	2638	4	3	Urban	1959	377998.5882
11	1121	5	2	Urban	2004	95961.92601
12	1466	5	3	Suburb	1951	191113.7687
13	2238	3	3	Suburb	1987	253358.645
14	1330	2	2	Suburb	1992	132172.3926
15	2482	4	3	Suburb	1989	231157.0277
16	1087	4	1	Urban	1976	118393.8232
17	2396	2	2	Suburb	1993	267377.3997
18	2123	5	2	Rural	1956	190773.1486
19	1871	4	2	Suburb	1977	172989.8049
20	2687	5	1	Urban	1979	239222.6678

Comparative Analysis

The objective of this section is to compare four regression algorithms for house price prediction using the same training and testing datasets.

The models evaluated are:

Linear Regression — Baseline regression model.

Ridge Regression — Linear model with regularization to control coefficient size.

Decision Tree Regressor — Tree-based model capable of learning nonlinear patterns.

Random Forest Regressor — Ensemble model that combines multiple decision trees.

```
#Import the additional algorithms
from sklearn.linear_model import Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

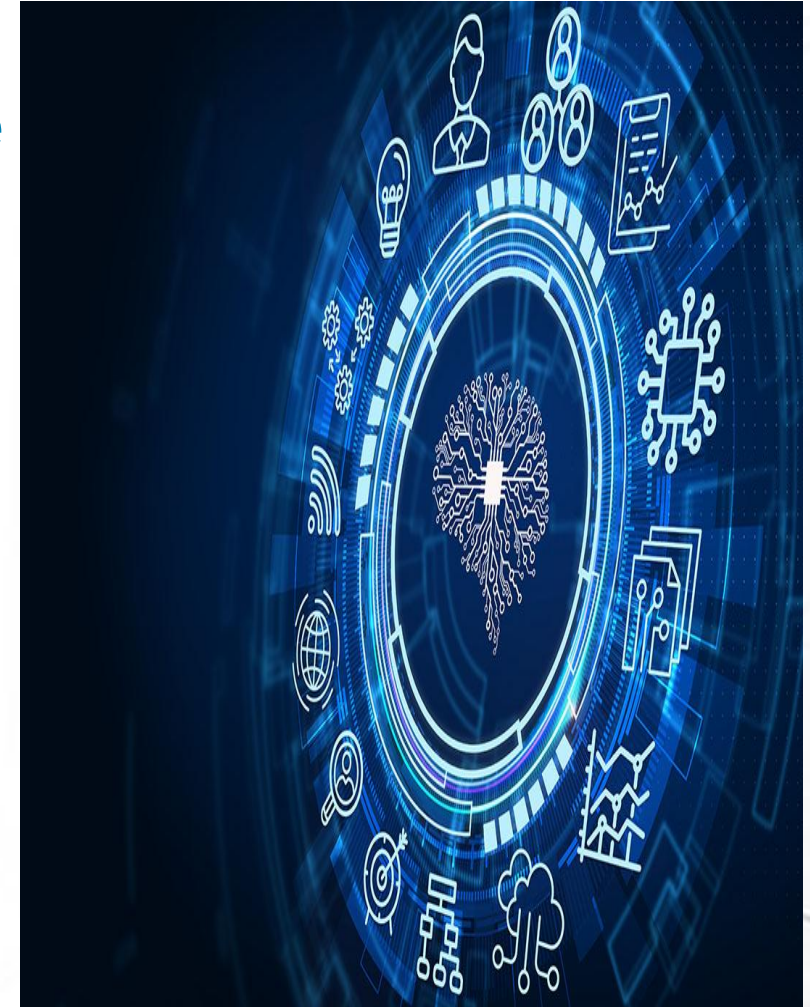
print("All model comparison libraries are ready!")
```

Train all four models

Display the model comparison table

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	39,517.15	2,445,804,887.80	49,455.08	0.5792
1	Ridge Regression	39,517.15	2,445,805,803.33	49,455.09	0.5792
2	Random Forest Regressor	40,470.29	2,561,218,317.80	50,608.48	0.5593
3	Decision Tree Regressor	42,426.82	2,836,706,617.21	53,260.74	0.5119

Lowest RMSE model: Linear Regression



Plot predictions from the best model

```
best_model_name = comparison_df.loc[0, "Model"]
best_predictions = predictions[best_model_name]

plt.figure(figsize=(8, 6))

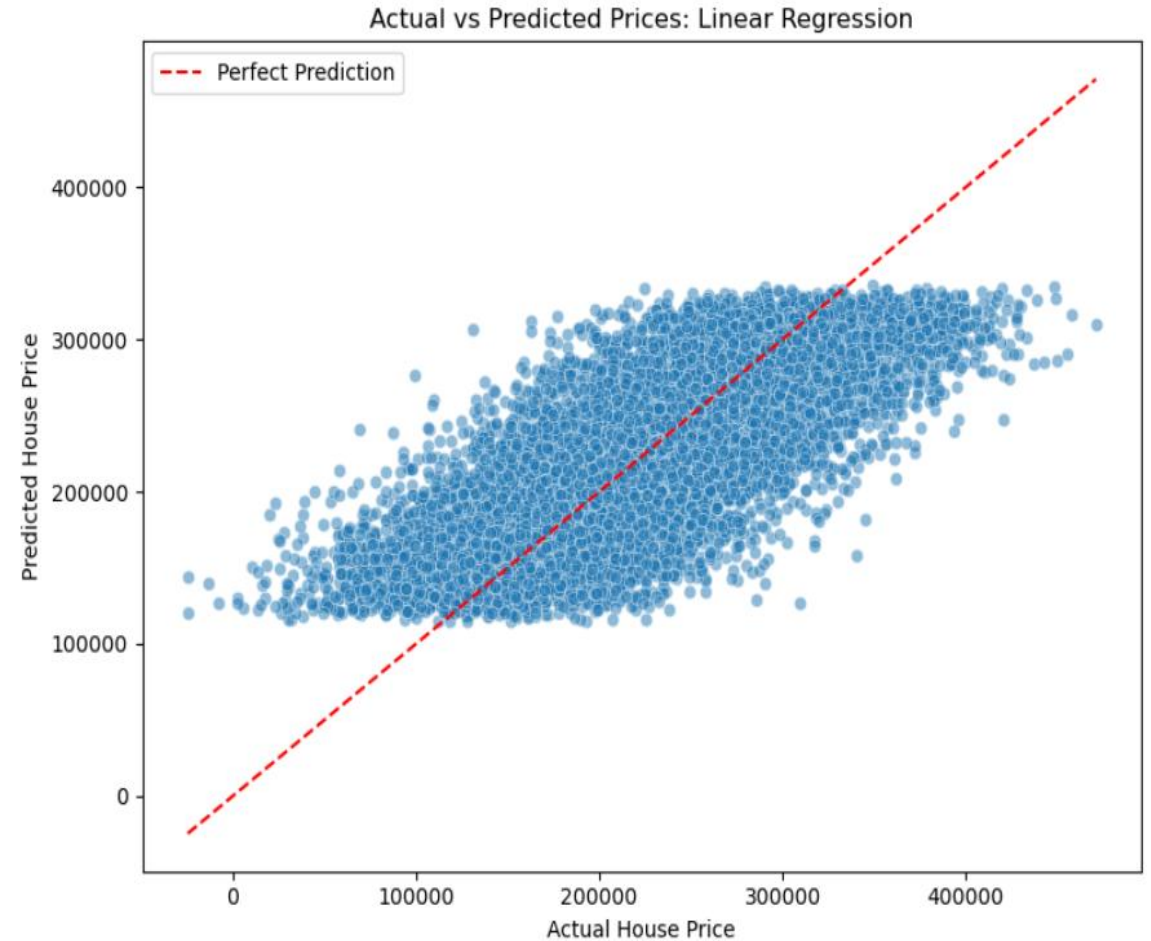
sns.scatterplot(
    x=y_test,
    y=best_predictions,
    alpha=0.5
)

min_price = min(y_test.min(), best_predictions.min())
max_price = max(y_test.max(), best_predictions.max())

plt.plot(
    [min_price, max_price],
    [min_price, max_price],
    "r--",
    label="Perfect Prediction"
)

plt.title(f"Actual vs Predicted Prices: {best_model_name}")
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.legend()
plt.tight_layout()
plt.show()

print("Model selected by lowest RMSE:", best_model_name)
```



Model selected by lowest RMSE: Linear Regression

Model Comparison — Key Observations

The four regression algorithms were evaluated using the same training and testing datasets to ensure a consistent comparison.

The evaluation focused on MAE, MSE, RMSE, and R^2 Score. Lower prediction errors and a higher R^2 Score generally indicate better predictive performance on the test dataset.

The model with the lowest RMSE was identified as the leading candidate based on that metric. However, the final selection should also consider MAE, R^2 Score, the difference between training and testing performance, and the reliability of the input data.

This comparison provides a foundation for selecting a suitable regression algorithm and identifying opportunities for further improvement.

What the metrics mean:

MAE-Average absolute prediction error, in the same units as house price

MSE-Average squared prediction error; larger errors are penalized more heavily

RMSE-Prediction error in the same units as price, with more sensitivity to large errors

R^2 -How much variation in test prices the model explains relative to a mean-price baseline



Conclusion

- In this project, I developed and compared four supervised machine learning regression algorithms for house price prediction: Linear Regression, Ridge Regression, Decision Tree Regressor, and Random Forest Regressor.
- The workflow included data inspection, categorical feature encoding, train-test splitting, model training, prediction, performance evaluation, and visualization. I evaluated the models using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score.
- Based on the test-set evaluation, Linear Regression emerged as the leading candidate among the four models, achieving an RMSE of 49,455.08, an MAE of 39,517.15, and an R^2 score of 0.5792. These results indicate that the model captures some of the variation in house prices, while a considerable portion remains unexplained.
- This project highlighted the importance of comparing multiple algorithms, selecting appropriate evaluation metrics, and examining prediction errors before choosing a final model. Data quality, particularly the presence of potentially invalid target values, should also be investigated to ensure reliable and meaningful predictions.
- Overall, this project strengthened my practical understanding of regression algorithms, model evaluation, comparative analysis, and data visualization using Python and Scikit-learn. It also provided hands-on experience with the end-to-end machine learning workflow and the challenges involved in building a reliable predictive model.

Future Improvements

1. **Data Quality:** Investigate negative target values, duplicate records, and unusual observations before finalizing the dataset.
2. **Exploratory Data Analysis:** Study feature distributions, relationships, and outliers to understand the factors associated with house prices.
3. **Feature Engineering:** Explore property age and additional reliable location or property-condition information, where available.
4. **Cross-Validation:** Use cross-validation to evaluate model stability across multiple data splits.
5. **Hyperparameter Tuning:** Optimize Ridge regularization and tree-based model parameters using cross-validation.
5. **Overfitting Analysis:** Compare training and testing performance to assess how well each model generalizes.
6. **Deployment:** Develop a simple Streamlit application to demonstrate house price predictions interactively.



Thank You

End Of Presentation